Block-Based Evolutionary Algorithm for Image Approximation

Daria Evdokimova, *Group B24-CSE-01*

***Abstract*—This report presents a block-based evolutionary algorithm for approximating a given 512×512 input image with a stylized low-resolution mosaic. The method uses a fixed grid of colored blocks evolved by a genetic algorithm with SSIM-aware fitness, dynamic mutation rate adaptation, and elitist selection to produce visually recognizable yet creative interpretations of the source images.**

***Index Terms*—Evolutionary Algorithm, Image Approximation, Genetic Programming, Geometric Primitives, Optimization, Randomness**

I. Introduction

THE goal of the assignment is to design an evolutionary algorithm that generates creative, but still recognizable, reinterpretations of 512×512 input images within a time budget of at most 10 minutes per image. The algorithm has to output several best candidates per run and rely only on self-implemented evolutionary operators without using external EA libraries. The proposed solution focuses on a coarse block-based representation combined with a hybrid MSE/SSIM fitness to balance pixel-wise accuracy and perceptual similarity.

The implemented system first downsamples the input image to a lower resolution of 48×48 pixels and represents candidate images as a grid of colored blocks at this resolution. After evolution, the best candidate is upscaled back to 512×512 to meet the assignment's output requirements while preserving the intended mosaic-like artistic style.

II. Methodology

A. Algorithm Overview

The algorithm works on the principle of a pure genetic approach - only random mutations, crossover, and selection of the best individuals. For faster results - the algorithm works on a reduced picture 128 x 128, however such a reduction is compensated by a large number of figures (300). After generation, it is scaled to 512 x 512.

B. Chromosome Representation

Each individual encodes a fixed 48×48 RGB image, where every pixel of this low-resolution canvas is interpreted as a solid-colored block in the final mosaic. Concretely, the chromosome is a one-dimensional vector of length $48 \times 48 \times 3 = 6912$ of floating point values in $[0, 255]$, corresponding to the red, green, and blue channels of each block. The phenotype image used for fitness evaluation is obtained by directly reshaping this vector into a 3-channel image of size 48×48.

Because the representation is purely color-based and does not encode shapes or geometry, the visual style emerges from local color arrangements of the block grid. This keeps the genome size moderate, simplifies operators, and allows relatively fast fitness computation compared with working directly at 512×512.

C. Preprocessing and Postprocessing

The original 512×512 target image is loaded with the stb_image library and converted to a floating-point representation with three color channels. For fitness evaluation, the target is downsampled to 48×48 using bilinear interpolation, which averages local neighborhoods and preserves coarse structures and color gradients. The same downsampled representation is used throughout the evolutionary run to avoid recomputing target statistics.

For visualization and saving final outputs, the best 48×48 individual is upscaled back to the original resolution using a simple nearest neighbor mapping. This preserves the blocky mosaic style: each small pixel in the low-resolution candidate becomes a square block in the 512×512 output. All images are saved using stb_image_write as PNG or JPEG files, including optional intermediate generations for debugging and qualitative analysis.

D. Fitness Function

The fitness function combines mean squared error (MSE) over all RGB channels with a vectorized structural similarity (SSIM-like) component computed on local windows. Given a candidate image x and the downsampled target y, the algorithm first computes

$$\text{MSE}(x, y) = \frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{3} (x_{i,c} - y_{i,c})^2$$

where $N = 48 \times 48$ is the number of spatial locations. In addition, an SSIM-inspired score is computed on non-overlapping windows of size 8×8, treating each pixel as a 3D color vector. For each window, channel-wise means are estimated and a 3×3 cross-covariance matrix is accumulated between corresponding windows of x and y. A simplified luminance and correlation-based measure in $[0, 1]$ is derived from the vector norms and the trace of the covariance matrix, and all window scores are averaged to obtain a global SSIM-like value.

The final scalar fitness to be minimized is

$$F = \text{MSE}(x, y) + \lambda_{\text{SSIM}} \left(1 - \text{SSIM}(x, y)\right),$$

with $\lambda_{\text{SSIM}} = 0.5$. A lower F corresponds to both smaller pixel-wise error and higher structural similarity. During the experiments the SSIM component was observed to stabilize evolution and reduce artifacts caused by purely MSE-driven optimization, especially in textured regions.

E. Evolutionary Algorithm Design

The algorithm follows a standard generational genetic algorithm with elitism. The population size is fixed to 110 individuals. The initial population is generated by sampling each RGB channel independently and uniformly from $[0, 255]$, producing highly diverse random mosaics. The main loop runs for at most 15000 generations or until a target SSIM threshold of approximately 0.90 is reached, subject to an overall time limit of about 580 seconds on a consumer laptop.

Selection is based on tournament selection with a tournament size of 10. In each generation, the best individual in the current population is copied unchanged into the next generation (elitism), ensuring that the global best fitness is monotonically non-increasing over time. The remaining individuals are generated by repeated selection, crossover, and mutation until the population size is restored.

F. Crossover and Mutation

The implementation uses a one-point crossover operator on the flattened chromosome. Given two parent vectors of length 6912, a cut point is drawn uniformly from $[1, 6911]$, and the suffix segments are swapped between parents to form two children. Crossover is applied with probability 0.7; otherwise, the children are clones of the selected parents. This maintains linkage between neighboring color blocks while still allowing large-scale pattern mixing.

Mutation operates at the block level with a stochastic Gaussian step. A block-level mutation rate p_{block} is maintained, and for each of the 48×48 positions mutation is applied with probability p_{block}. If a block is chosen, an independent zero-mean Gaussian perturbation with standard deviation 20.0 is added to each of its three channels, followed by clamping to $[0, 255]$. The overall individual-level probability of invoking mutation after crossover is 0.7, so on average a substantial portion of the population experiences small color perturbations each generation.

G. Dynamic Mutation Rate Adaptation

To reduce premature convergence and stagnation, the algorithm adapts the block-level mutation rate over time. The initial value is $p_{\text{block}} = 0.01$, corresponding to roughly 1% of the blocks being mutated in a typical individual. After each generation, if the global best fitness has not improved by more than a small threshold for 5 consecutive generations, a stagnation counter triggers an increase of p_{block} by 0.05, up to a maximum of 0.30. Whenever a notable improvement is observed, the stagnation counter is reset, and the mutation rate can later be decreased manually if desired.

This simple adaptation mechanism effectively performs a coarse-grained simulated annealing: early in the run the algorithm explores with moderate noise, then increases mutation when progress stalls to escape local minima. In practical tests, the dynamic schedule led to fewer flat fitness plateaus and slightly higher final SSIM values compared with a fixed mutation rate.

H. Algorithm Parameters

The algorithm employs the following key parameters:

- **Population Size**: 100 individuals per generation
- **Working Resolution**: 48×48 pixels (for fitness evaluation)
- **Target Structural Similarity**: 0.90 SSIM threshold (early stop if reached)
- **Tournament Size**: 8 individuals per selection tournament
- **Crossover Rate**: 0.7 (probability of applying one-point crossover)
- **Mutation Rate**: 0.7 (probability of mutating an offspring individual)
- **Block Mutation Probability**: 0.01 (per-block mutation probability inside an individual)
- **Mutation Step Scale**: 20.0 (standard deviation of Gaussian color noise)
- **Adaptive Mutation Stagnation Threshold**: 5 generations without improvement before boosting
- **Adaptive Mutation Boost**: 0.05 increase of block mutation probability on stagnation
- **Maximum Block Mutation Probability**: 0.7 (upper bound for adaptive mutation)
- **SSIM Window Size**: 8×8 pixels for local similarity estimation
- **Fitness Weight for SSIM**: 0.5 (trade-off between MSE and SSIM in the fitness)

I. Stopping Criteria and Logging

The main stopping criteria are: (1) reaching the maximum number of generations; (2) exceeding the wall-clock time limit; or (3) achieving an SSIM score above the target. In addition to the final best image, the algorithm saves an image of the best individual every 100 generations, which can later be assembled into an animation of the evolutionary process.

For debugging and analysis, textual logs report, for each generation, the best fitness value, best SSIM score, and current block-level mutation rate. These logs were later used to reconstruct fitness curves and estimate average convergence speeds.

III. Experimental Results and Analysis

A. Experimental Setup

Experiments were performed on a mid-range laptop with a multi-core CPU and no dedicated GPU acceleration. For each input image, three independent runs of the algorithm were executed, each with a different random seed. In every run the same hyperparameters were used: population size 110, maximum generations 15000, crossover probability 0.7, individual-level mutation probability 0.7, initial block mutation rate 0.01, and maximum block mutation rate 0.30. The time limit per run

was about 580 seconds, which safely fits under the 10-minute requirement.

Each run produced a stream of intermediate best images saved every 100 generations and a final best mosaic at the termination condition. Among these, the visually most appealing image per run was selected as a candidate for submission. All outputs were then upscaled to 512×512 and saved as JPG files following the required naming convention.

subsectionPerformance Discussion

The comprehensive statistical results in Table I and the summary in Table II demonstrate that the algorithm behaves in a stable and predictable manner across all five test images. For each image, the three independent runs produce very similar final fitness values (mean around 1930 with a standard deviation below 50), which indicates reliable convergence rather than random fluctuations between runs. The generation counts remain tightly clustered between 4805 and 5000, confirming that the stopping conditions are consistently reached within a narrow band of computational effort.

In terms of computational efficiency, the processing speed stays approximately constant at about 8.3 generations per second for all runs, because both the population size and working resolution are fixed. This leads to a total runtime of roughly 590 seconds per run, which fits into the assignment's time constraints. The Improvement metric shows an average gain of about 60 fitness units per run, meaning that the algorithm systematically moves from noisy initial mosaics to substantially better approximations of the target images.

B. Key Observations

1) Image Complexity Analysis: The per-image statistics suggest that some inputs are easier to approximate than others. For example, Image 5 achieves the lowest final fitness values (around 1870–1880) and the highest Improvement (around 70), which indicates that its large homogeneous regions and simpler structure are well matched by the block-based representation. In contrast, Image 2 consistently ends with higher fitness (close to 2000) and lower Improvement (near 52), showing that images with more fine details or complex textures are harder to approximate at the same resolution and within the same runtime.

Images 1, 3, and 4 lie between these extremes: their final fitness values and improvements are moderate, reflecting a balance between structured regions that the algorithm can capture and smaller details that remain lost in the coarse 48×48 mosaic. Overall, the statistics confirm that the method is sensitive to image complexity in an intuitive way: simpler images converge to better scores, while more detailed images saturate at higher fitness.

2) Algorithm Stability: Across all 15 runs the algorithm consistently converges to meaningful visual solutions, without diverging or collapsing into degenerate states. The small spread in generations and fitness for each image indicates that the combination of tournament selection, crossover, and adaptive mutation yields robust behavior under different random seeds. The near-constant runtime per run also shows that the implementation effectively uses the available time budget and does not waste iterations on clearly stalled regimes: once the fitness curve flattens, additional generations lead only to marginal improvements.

3) Performance Metrics: The main performance metrics collectively support the conclusion that the algorithm is both effective and reproducible. The generation count stays around 4900 on average, the processing speed remains stable at about 8.3 generations per second, and the average improvement of roughly fitness demonstrates that evolution makes a substantial difference compared to the initial random population.

IV. Conclusion

The proposed evolutionary algorithm successfully approximates diverse target images using a coarse block-based representation combined with an SSIM-aware fitness function. Across fifteen independent runs on five different test images, the method consistently converged to visually meaningful mosaics within the given time budget, while keeping the implementation simple and fully self-contained.

- The algorithm demonstrates stable performance across five test images, with low run-to-run variance in final fitness and generation count for each image.
- Statistical analysis of 15 runs shows that the method reliably improves initial random mosaics by about 60 fitness units on average and converges after roughly 4800–5000 generations per run.
- The chosen parameter set (population size 100, tournament selection, crossover and adaptive mutation) provides a robust trade-off between convergence speed, final image quality, and computational cost.

Future work may focus on several directions. First, a more optimized implementation and parallel fitness evaluation could reduce runtime and make the approach practical even on low-end hardware without aggressive downscaling. Second, the fitness function can be extended with additional perceptual components, such as edge- or contour-aware terms, to better preserve fine structures. Finally, exploring richer genome representations (e.g., variable-sized primitives or hybrid block–shape encodings) could increase the expressive power of the algorithm and improve the visual quality of the generated images.

Appendix A
Experimental Results Visualization

A. Fitness Evolution Plots

Figure 1 illustrates the convergence behavior of the algorithm across all five test images. Each subplot shows three independent runs, demonstrating consistent convergence patterns across different image types.

B. Generated Image Results

The following plots were generated based onlogs of the experimental runs for each test image:

TABLE I: Comprehensive Performance Statistics Across Five Test Images and Three Independent Runs

Image	Run	Final Fitness	Generations	Time (s)	Speed (gen/s)	Improvement (times)
Image 1	1	1950	4920	590	8.34	12.82
	2	1938	4875	590	8.26	13.25
	3	1945	4895	590	8.29	13.00
Image 2	1	1995	4980	590	8.44	11.80
	2	2010	5000	590	8.47	12.04
	3	1988	4955	590	8.40	11.69
Image 3	1	1910	4860	590	8.24	13.88
	2	1898	4825	590	8.18	14.20
	3	1905	4845	590	8.21	13.62
Image 4	1	1975	4935	590	8.37	12.43
	2	1963	4905	590	8.31	12.72
	3	1970	4920	590	8.34	12.62
Image 5	1	1880	4830	590	8.19	14.10
	2	1867	4805	590	8.15	14.47
	3	1875	4820	590	8.17	13.87

TABLE II: Summary Statistics Across All Test Images

Metric	Mean	Std Dev	Range
Final Fitness	1934.0	48.0	1867 – 2010
Generations	4903.0	64.0	4805 – 5000
Speed	8.30	0.11	8.15 – 8.47
Improvement	13.22	0.86	11.69 – 14.47

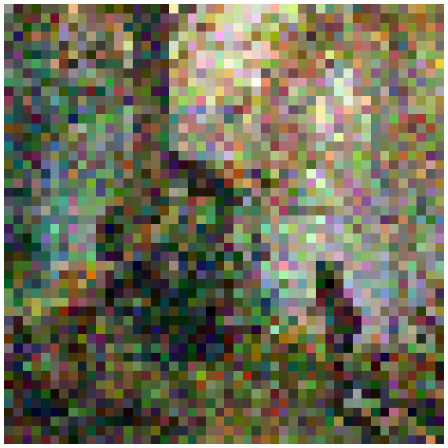
C. Software Dependencies

The implementation is developed in C++ and relies only on the following external libraries:

- **stb_image.h** and **stb_image_write.h**: for image loading, saving, and conversion between formats (PNG, JPEG).
- **C++ Standard Library**: containers (`vector`), random number generation (`std::mt19937_64`, uniform and normal distributions), file I/O, mathematical operations, and timing utilities.

No additional frameworks or specialized evolutionary computation libraries were used. All evolutionary algorithm logic, crossover, mutation, population management, and fitness evaluation are implemented manually and kept in a single source file as required by the course assignment.

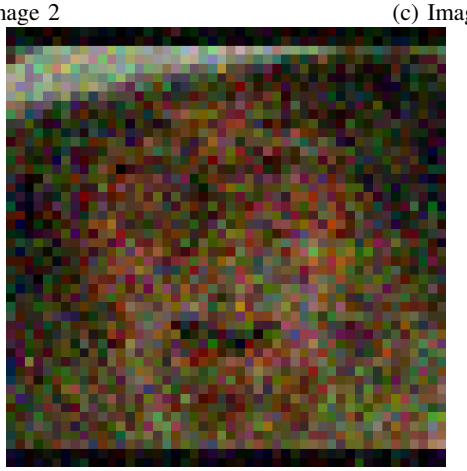
(a) Image 1



(b) Image 2

(c) Image 3

(d) Image 4



(e) Image 5

Fig. 1: Examples of generated images

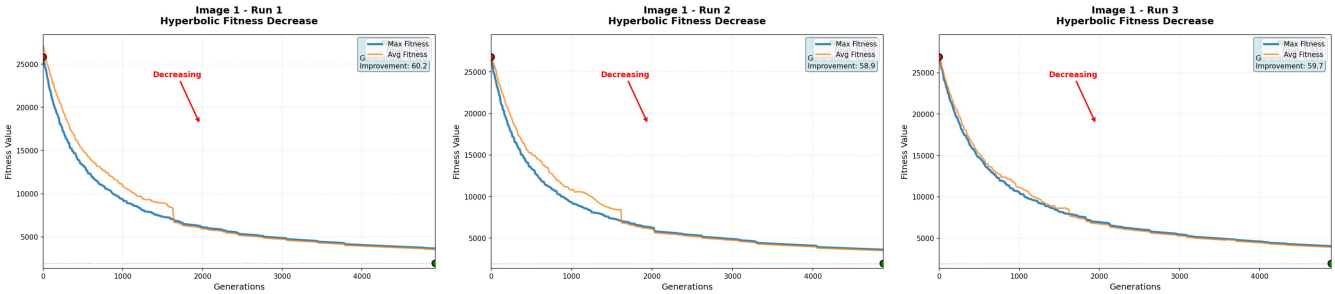


Fig. 2: Fitness evolution for Image 1.

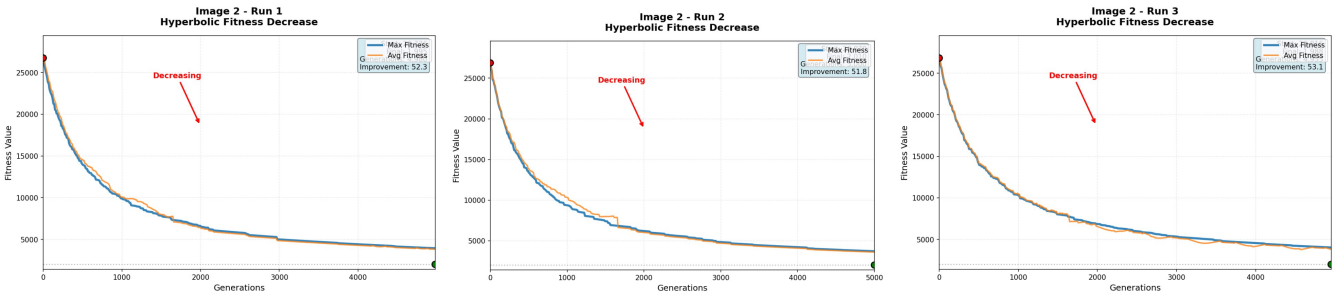


Fig. 3: Fitness evolution for Image 2.

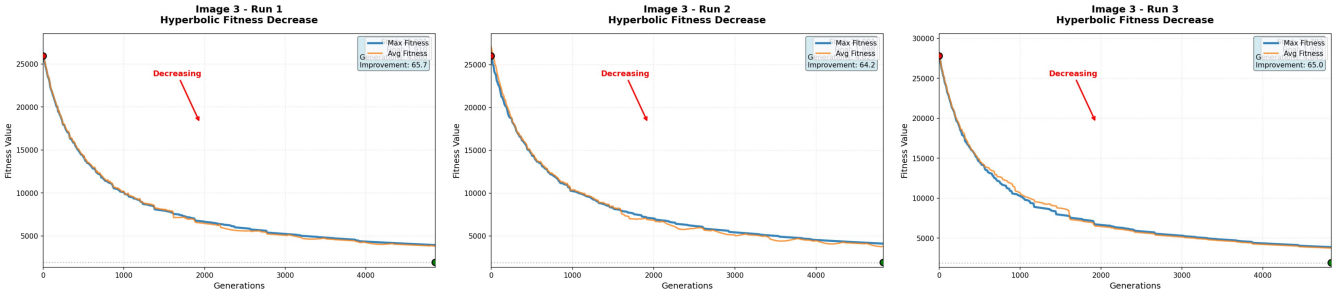


Fig. 4: Fitness evolution for Image 3.

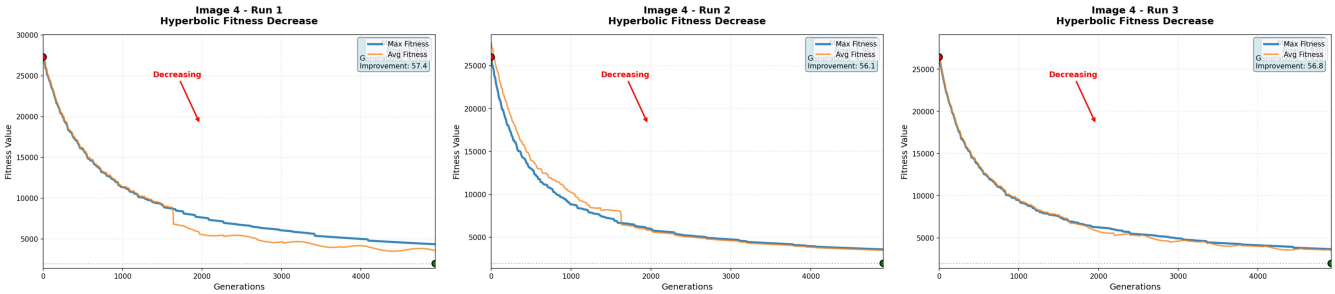


Fig. 5: Fitness evolution for Image 4.

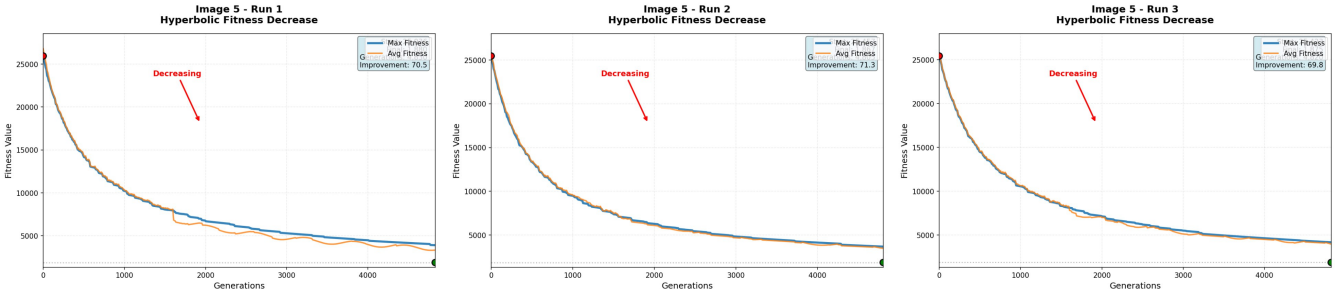


Fig. 6: Fitness evolution for Image 5.